

PROCEDIMIENTO Y DOCUMENTACIÓN CONSOLIDADA DE PRUEBAS

Servicios backend — RentaMóvil

RENTAMOVIL

Thiago Fabian Rojas Guevara — David Santiago Erez Rodríguez — Nicole Dayana Ramírez Vargas

ANÁLISIS Y DESARROLLO DE SOFTWARE 3145556

SERVICIO NACIONAL DE APRENDIZAJE

2026

Project	RentaMóvil — five Java backend modules
Date of preparation	October 9, 2026
Version	1.0
Prepared by	Development team
Reviewed by	[Name of reviewer]
Approved by	[Name of leader or person responsible]
Status	Testing baseline executed; gaps identified

1. CHANGE CONTROL

Version	Date	Description	Responsible
1.0	2026-10-09	Consolidation of the procedure, inventory, and test results for the five Java backend modules.	Development team

2. OBJECTIVE

Establish a common, repeatable, and verifiable procedure to design, implement, execute, and record testing for RentaMóvil backend services. This document consolidates the current inventory, results per module, observed limitations, and recommended actions.

Existing and verified capabilities are documented. Suggested tests are presented as pending improvements, not as already implemented coverage.

3. SCOPE

This document covers the five Java backend modules:

- **rtm-iam:** identity and access.
- **rtm-fleet-maintenance:** fleet and maintenance.
- **rtm-booking-reservation:** bookings and rentals.
- **rtm-payment-billing:** payments and billing.
- **rtm-api-gateway:** request entry and routing.

It includes unit tests for domain and use cases, direct testing of web components/filters, a JPA repository suite with H2 in Fleet, and a separate reference to manual integration verification using Postman.

It does not certify all HTTP endpoints, full compatibility with SQL Server, contracts between services, performance, comprehensive security, or automated E2E flows. Web and mobile applications, as well as database migration repositories, are not part of this automated baseline.

4. COMPONENTS AND TEST ENVIRONMENT

Module	Java	Spring Boot	Production Persistence
rtm-iam	21	4.1.1	JPA / SQL Server
rtm-fleet-maintenance	21	4.1.1	JPA / SQL Server
rtm-booking-reservation	21	4.1.1	JPA / SQL Server
rtm-payment-billing	21	4.1.1	JPA / SQL Server
rtm-api-gateway	21	4.0.8	Does not manage business persistence

The primary verified tool versions are Java 21 and Maven 3.9.16. The modules use JUnit Jupiter via spring-boot-starter-test; Mockito and AssertJ appear in existing tests. Fleet adds the test JPA starter and H2 in test scope.

Execute with JDK 21, which is the version declared in the POMs. In this environment, an execution with JDK 27 failed during the initialization of the Lombok annotation processor; the baseline was obtained using JDK 21.

5. TESTING STRATEGY

5.1 Domain and use cases

Test business rules using JUnit 5, isolating external dependencies with doubles where appropriate, and verifying relevant results, errors, and interactions. Do not spin up Spring for unit tests of logic that can be tested directly.

5.2 Controllers and filters

Some tests invoke controller methods directly or test filters as isolated units. They serve to validate delegation and local logic, but they do not traverse the MVC server nor do they verify HTTP routes, JSON serialization, input validation, or integrated HTTP handling on their own.

No @WebMvcTest/MockMvc tests were found in the five modules. If HTTP contracts need to be validated, focused MVC tests must be added.

5.3 Persistence

Fleet contains a @DataJpaTest suite with H2 for branch, vehicle, maintenance, and model queries. H2 validates JPA and JPQL behavior, but it does not substitute SQL Server nor does it cover all its specific details.

IAM, Booking, and Billing use JPA in production, but no @DataJpaTest, H2, or Testcontainers tests were found in their current suites. Their repositories, mappings, and queries are not considered automatically verified by this baseline.

The Fleet JPA suite passes, although Hibernate logs a warning when attempting to create shared.audit because the shared schema is not present in H2. The audit and that schema remain outside the suite's coverage.

5.4 System integration

The Fleet guide documents a manual verification with Docker, SQL Server, gateway, and Postman: 37 out of 37 requests were successful on October 4, 2026. This is dated manual evidence; it is not part of mvn test nor was it repeated in the automated execution on October 9.

6. PROCEDURE

6.1 Analyze the change

1. Inspect the POM, package structure, source classes, and nearby tests.
2. Identify existing behavior, contracts, dependencies, and actual exceptions.
3. Select the corresponding test level; do not invent entities, endpoints, or responses.
4. Determine the boundaries of what the test can prove, especially if it utilizes mocks or an embedded database.

6.2 Implement tests

1. Place tests under src/test/java, respecting the package of the code being tested.
2. Isolate use cases from Spring and use mocks for external dependencies when appropriate.
3. Test web components directly only for their local logic; use MockMvc if HTTP guarantees are required.
4. For JPA repositories, use a persistence slice with minimal data and document the difference compared to SQL Server.
5. Cover successful paths, alternate paths, and errors relevant to the change, using assertions on values and interactions.

6.3 Execute from the rentacar root

The repositories are independent Maven projects; there is no common aggregator POM. Execute each module separately:

```
mvn -f rtm-iam/pom.xml test
mvn -f rtm-fleet-maintenance/pom.xml test
mvn -f rtm-booking-reservation/pom.xml test
mvn -f rtm-payment-billing/pom.xml test
mvn -f rtm-api-gateway/pom.xml test
```

To execute a specific suite, add -Dtest=TestName before test. Maven Surefire reports are located in target/surefire-reports inside each module.

6.4 Evaluate and record

1. Confirm compilation and BUILD SUCCESS.
2. Record the number of tests, failures, errors, and skipped tests per module.
3. Investigate initialization warnings even if they do not change the suite's outcome.
4. Record date, Java version, command, branch/commit, and test scope.
5. Separate automated evidence from manual tests and describe their limitations.

7. INVENTORY AND RESULTS BY MODULE

Module	Inventory and exercised behavior	Verified result on 2026-10-09
rtm-iam	Ten test classes, focused on use cases: registration, login/logout, tokens, profile, email, password, roles, and status.	34 tests; 0 failures; 0 errors; BUILD SUCCESS
rtm-fleet-maintenance	Use cases, controllers via direct invocation, JWT verification, and JPA queries with H2.	114 tests; 0 failures; 0 errors; 0 skipped; BUILD SUCCESS
rtm-booking-reservation	Reservation/rentals/insurance domain, payment events, expiration, and directly invoked controllers. Two in-memory repositories serve as test utilities, not suites.	53 tests; 0 failures; 0 errors; BUILD SUCCESS
rtm-payment-billing	Two test classes: Payment entity and PaymentUseCaseImpl use case.	14 tests; 0 failures; 0 errors; BUILD SUCCESS
rtm-api-gateway	Three test classes: JWT filter, correlation ID filter, and in-memory rate limiter.	10 tests; 0 failures; 0 errors; BUILD SUCCESS
Total Java backend	Independent executions per module	225 tests; 0 failures; 0 errors

The sum of 225 is an aggregate of five independent Maven executions, not a multi-module reactor execution.

8. ACCEPTANCE CRITERIA

- The affected module compiles with Java 21 and its complete suite finishes without failures or errors.
- Names and scenarios correspond to existing classes and behavior.
- Tests are repeatable and do not depend on remote services or real credentials, except in controlled manual procedures.
- A slice test declares the database engine and differences regarding production.
- Manual checks are not presented as automated ones.
- No coverage percentage is reported if a configured measurement does not exist.

9. KNOWN GAPS AND RISKS

Priority	Gap	Affected Scope
High	Lacks real HTTP validation with @WebMvcTest/MockMvc; direct tests do not test transport or serialization.	All five modules; especially Fleet and Booking controllers
High	No repository tests in three JPA modules; Fleet uses H2, not SQL Server, and does not test shared.audit.	IAM, Booking, Billing, and Fleet boundaries
High	Billing has two test classes and 14 cases, the smallest inventory per service.	Billing
Medium	No automated tests for Gateway routes/upstreams in an HTTP context.	API Gateway
Medium	JaCoCo is not configured, so there is no reproducible coverage percentage.	All
Medium	No CI pipeline configured to execute suites when integrating changes.	All
Medium	Postman integrated verification is manual, not repeatable via mvn test.	Entire system

10. RECOMMENDED IMPROVEMENT ACTIONS

1. Prioritize testing business rules and payment errors in Billing.
2. Add MVC tests with MockMvc for endpoints and HTTP error handling.
3. Add persistence tests for IAM, Booking, and Billing; evaluate Testcontainers with SQL Server for actual compatibility.
4. Complete routing and upstream testing for the Gateway.
5. Configure JaCoCo with agreed goals and add mvn test to CI per repository.
6. Gradually automate the most critical Postman/E2E flows.

Note: These actions are proposals; they are not part of the results executed here.

11. ROLES AND RESPONSIBILITIES

Role	Responsibility
Developer	Implement tests for the change, execute them in their module, and record the results.
Technical Reviewer	Confirm that scenarios correspond to the actual code and that its limitations are declared.
Quality Manager or Leader	Approve acceptance criteria and prioritize gaps.

Role	Responsibility
Environment Responsible	Maintain the required test services and data for manual or integrated validations.

12. RELATED DOCUMENTS

- Procedimiento_Pruebas_Software.md: detailed evidence and Fleet JPA considerations.
- Documentacion_Pruebas_Otros_Servicios.md: detailed inventory of IAM, Booking, Billing, and Gateway.
- 11-guia-ejecucion-y-pruebas-fleet.md: local execution and manual verification with Docker and Postman.
- 09-buenas-practicas.md: practices applied in the project.
- Procedimiento_Implementacion_Software.docx: system deployment procedure.